

HOSPITAL MANAGEMENT SYSTEM

Database Design Project — Milestones 2 to 5

Course: Database Management Systems (DBMS)
Project: Hospital Management System (HMS)
Milestone: 2 — Normalization | 3 — Data | 4 — DDL | 5 — Population
Tool Used: MySQL Workbench 8.0 + Python Faker
Deadline: 17 May 2026
Format: GroupName_Version_Control_Doc_DBLab.pdf

M2

Normalize

M3

Dataset

M4

DDL

M5

Populate

Table of Contents

Milestone 2	Schema Normalization (1NF → 3NF)	Pg. 3
Milestone 3	Dataset Preparation & Dataflow Description	Pg. 8
Milestone 4	CREATE TABLE DDL with All Constraints	Pg. 13
Milestone 5	Data Population & Validation Queries	Pg. 18
Appendix A	GitHub Commit Log Structure	Pg. 22
Appendix B	Entity List Summary	Pg. 23

Milestone 2

Schema Normalization

Justifying 1NF → 2NF → 3NF compliance for all 14 HMS tables

Milestone 2 — Schema Normalization (1NF to 3NF)

Normalization is the process of organizing a relational database to reduce data redundancy and improve data integrity. The HMS schema has been designed to satisfy Third Normal Form (3NF) across all 14 tables. Below is a systematic justification for each normal form.

2.1 What is Normalization?

Normalization eliminates three types of anomalies from a relational database: **Insertion anomalies** (cannot insert data without unrelated data), **Update anomalies** (changing one fact requires updates in multiple places), and **Deletion anomalies** (deleting a row unintentionally destroys other facts). The HMS schema has been structured to eliminate all three.

2.2 First Normal Form (1NF)

Definition

A table is in 1NF if: (1) each cell contains a single atomic value, (2) each row is uniquely identifiable, and (3) there are no repeating groups.

How HMS Satisfies 1NF

- Every column holds one and only one value — no comma-separated lists, no arrays.
- Every table has a single-column INT AUTO_INCREMENT primary key (surrogate key).
- The many-to-many relationship between PRESCRIPTION and MEDICINE is broken into the PRESCRIPTION_DETAIL bridge table — eliminating the repeating group of medicines that would otherwise exist inside a prescription row.
- Patient addresses are stored as TEXT (single address). Multiple addresses would require a separate ADDRESS table (not required for this scope).
- Doctor specializations are stored atomically. A doctor with two specializations would need a DOCTOR_SPECIALIZATION table (single specialization is the business rule here).

2.3 Second Normal Form (2NF)

Definition

A table is in 2NF if it is in 1NF AND every non-key attribute is fully functionally dependent on the entire primary key (no partial dependencies).

How HMS Satisfies 2NF

All 14 tables use single-column surrogate primary keys (INT AUTO_INCREMENT). Since there are no composite primary keys, partial dependencies are structurally impossible — every non-key column must depend on the one-column PK by definition. This design choice was deliberate to enforce 2NF automatically.

- PRESCRIPTION_DETAIL (detail_id PK): dosage, frequency, duration depend on the specific prescription-medicine combination, not just on prescription_id or medicine_id alone — correctly placed here.

- APPOINTMENT (appointment_id PK): reason and notes describe this specific appointment, not the patient or doctor in isolation.
- BILLING (bill_id PK): total_amount and payment_status describe this specific bill, not the patient generically.

2.4 Third Normal Form (3NF)

Definition

A table is in 3NF if it is in 2NF AND there are no transitive dependencies (non-key attributes must not depend on other non-key attributes).

Key 3NF Design Decisions

→ DOCTOR stores department_id (FK), not department_name

If department_name were stored in DOCTOR, then: doctor_id → department_id → department_name. That is a transitive dependency. By storing only the FK, DOCTOR is in 3NF. Department name lives in DEPARTMENT.

→ BILLING stores patient_id and admission_id separately

A billing row needs the patient reference independently of the admission (outpatient bills exist without admissions). Patient name is NOT stored in BILLING — it is fetched via JOIN with PATIENT, eliminating the transitive: bill_id → patient_id → patient_name.

→ MEDICAL_RECORD stores doctor_id, not doctor name or specialization

doctor_id is the FK. Doctor name and specialization live in DOCTOR. No transitive dependency.

→ NURSE extends STAFF without duplicating staff columns

Nurse-specific attributes (shift, certification) live in NURSE. Staff attributes (name, phone) live in STAFF. The IS-A relationship is modeled correctly — no duplication, no transitive dependency.

→ PRESCRIPTION_DETAIL does not store medicine_name or patient_id

medicine_name is in MEDICINE. Patient is reachable via prescription → medical_record → patient. We store patient_id in PRESCRIPTION as a denormalization shortcut for query performance (acceptable).

2.5 Normalization Justification Table (All 14 Tables)

Table	NF Satisfied	Justification
department	1NF ✓ 2NF ✓ 3NF ✓	Atomic values; single PK; no transitive deps — dept_name doesn't depend on head_doctor_id
doctor	1NF ✓ 2NF ✓ 3NF ✓	Single PK; department_name not stored here (prevents transitive dep via dept_id)
patient	1NF ✓ 2NF ✓ 3NF ✓	Atomic address; single PK; all columns depend only on patient_id
staff	1NF ✓ 2NF ✓ 3NF ✓	Single PK; role is atomic; dept_name not stored here
nurse	1NF ✓ 2NF ✓ 3NF ✓	IS-A pattern avoids duplicating STAFF columns; shift/cert are nurse-specific facts
room	1NF ✓ 2NF ✓ 3NF ✓	room_type and price_per_day depend directly on room_id, not on dept
appointment	1NF ✓ 2NF ✓ 3NF ✓	No patient/doctor data duplicated; status/reason are appointment-specific facts
medical_record	1NF ✓ 2NF ✓ 3NF ✓	Diagnosis/treatment depend on record_id; doctor info not duplicated

prescription	1NF ✓ 2NF ✓ 3NF ✓	Header record only; medicine details in PRESCRIPTION_DETAIL
medicine	1NF ✓ 2NF ✓ 3NF ✓	All attributes (name, price, stock) are facts about this medicine only
prescription_detail	1NF ✓ 2NF ✓ 3NF ✓	Resolves M:N; dosage/frequency are facts of this medicine ON this prescription
admission	1NF ✓ 2NF ✓ 3NF ✓	Dates/diagnosis are admission-specific; patient/room/doctor data via FK joins
billing	1NF ✓ 2NF ✓ 3NF ✓	Financial facts depend on bill_id; patient/admission details via FK
lab_test	1NF ✓ 2NF ✓ 3NF ✓	Results/status are test-specific; patient and doctor referenced via FK

2.6 Updated ERD — Post-Normalization Notes

The ERD submitted in Milestone 1 already reflects the normalized schema. No structural changes were required during normalization review because the design was built normalization-first. Key structural decisions that enforce normalization: (1) PRESCRIPTION_DETAIL bridge table, (2) NURSE extends STAFF via FK (not duplicate columns), (3) DEPARTMENT.head_doctor_id uses FK not stored name, (4) All FKs reference IDs only — names/labels are never duplicated across tables.

2.7 Relationship Summary Table

From Table	To Table	Cardinality	FK Column	Description
DEPARTMENT	DOCTOR	1 : Many	department_id	A dept employs many doctors
DEPARTMENT	ROOM	1 : Many	department_id	A dept contains many rooms
DEPARTMENT	STAFF	1 : Many	department_id	A dept includes many staff
PATIENT	APPOINTMENT	1 : Many	patient_id	Patient books many appointments
DOCTOR	APPOINTMENT	1 : Many	doctor_id	Doctor conducts many appointments
PATIENT	MEDICAL_RECORD	1 : Many	patient_id	Patient has many records
DOCTOR	MEDICAL_RECORD	1 : Many	doctor_id	Doctor writes many records
APPOINTMENT	MEDICAL_RECORD	1 : 0..1	appointment_id	Appt may generate 1 record
MEDICAL_RECORD	PRESCRIPTION	1 : Many	record_id	Record leads to prescriptions
PATIENT	PRESCRIPTION	1 : Many	patient_id	Patient receives many Rx
PRESCRIPTION	PRESCRIPTION_DETAIL	1 : Many	prescription_id	Rx has many medicine lines
MEDICINE	PRESCRIPTION_DETAIL	1 : Many	medicine_id	Medicine on many Rx lines
PATIENT	ADMISSION	1 : Many	patient_id	Patient admitted multiple times
DOCTOR	ADMISSION	1 : Many	doctor_id	Doctor oversees many admissions

ROOM	ADMISSION	1 : Many	room_id	Room assigned to many admissions
PATIENT	BILLING	1 : Many	patient_id	Patient receives many bills
ADMISSION	BILLING	1 : 0..1	admission_id	Admission generates one bill
STAFF	NURSE	1 : 0..1	staff_id	Staff member may be a nurse
PATIENT	LAB_TEST	1 : Many	patient_id	Patient undergoes many tests
DOCTOR	LAB_TEST	1 : Many	doctor_id	Doctor orders many tests
MEDICAL_RECORD	LAB_TEST	1 : Many	record_id	Record includes many tests

Milestone 3

Dataset Preparation & Dataflow Description

Synthetic data generated using Python Faker library — 14 CSV files ready for import

Milestone 3 — Dataset Preparation

3.1 Data Strategy

Since this is a course project, synthetic (dummy) data has been generated using the **Python Faker library** and Python's built-in **random** module. All data is structurally valid, referentially consistent, and statistically realistic for a mid-sized hospital.

Tool Used: Python 3.12 + Faker 24.x
Seed: random.seed(42) and Faker.seed(42) — fully reproducible
Output: One CSV file per table (14 total)
Total Records: 1,158 rows across all tables

3.2 Dataset Summary Table

Table	Module	Rows Generated	Tool Used	Notes
department	Organization	15	Manual	Real hospital departments
doctor	Staff	40	Faker	Auto-generated names, licenses
patient	Patient Mgmt	100	Faker	Realistic Pakistani phone numbers
staff	HR	50	Faker	10 distinct job roles
nurse	HR	25	Random sample	IS-A subset of staff
room	Inpatient	40	Random	5 room types, realistic pricing
appointment	Scheduling	200	Faker + Random	60% Completed, 30% Scheduled
medical_record	Clinical	120	Faker	Linked to completed appointments
prescription	Pharmacy	120	Derived	From medical records
medicine	Pharmacy	50	Manual	Real Pakistani market medicines
prescription_detail	Pharmacy	228	Random	1-3 medicines per prescription
admission	Inpatient	60	Random	80% discharged, 20% active
billing	Finance	60	Derived	Linked to admissions
lab_test	Laboratory	100	Random	16 real lab test types

3.3 Data Generation Rules

Referential Integrity

Foreign keys were enforced during generation: Doctor department_ids are drawn from the 15 valid department IDs. Appointments reference only valid patient_ids (1-100) and doctor_ids (1-40). Medical records are generated only from Completed appointments. Prescriptions are derived from medical records. Prescription details reference only valid prescription_ids and medicine_ids.

Business Rules Applied

- Appointment status distribution: 60% Completed, 30% Scheduled, 10% Cancelled — realistic hospital utilization
- Admission discharge rate: 80% discharged, 20% still admitted — reflects active inpatient census
- Billing collection rate: ~70% fully paid, remainder partial or unpaid — typical healthcare receivables
- Lab test completion: randomized Pending/Completed status with results only for Completed tests
- Phone numbers follow Pakistani mobile format (03xxxxxxxxx) to match project context
- Doctor license numbers follow PMC (Pakistan Medical Commission) format: PMC-XXXXX
- Medicine data reflects actual drugs available in Pakistani hospital formularies

3.4 Dataflow Description

The following describes how data flows through the Hospital Management System, from patient entry to billing settlement.

1	Patient Registration Flow: patient A new patient visits the hospital. Their personal information (name, DOB, gender, blood type, contact) is recorded in the PATIENT table. A unique patient_id is assigned.
2	Department & Doctor Assignment Flow: department → doctor Doctors are assigned to departments. The DEPARTMENT table records each unit; DOCTOR references department_id as a FK. Department heads are updated once doctors are created.
3	Appointment Booking Flow: patient → appointment ← doctor The patient books an appointment with a doctor. The APPOINTMENT table records the scheduled datetime, reason, and status (Scheduled/Completed/Cancelled).
4	Clinical Visit & Medical Record Flow: appointment → medical_record When the appointment is completed, the doctor records the diagnosis and treatment in MEDICAL_RECORD. This record references the appointment and the patient.
5	Prescription Issuance Flow: medical_record → prescription → prescription_detail ← medicine The doctor issues a PRESCRIPTION linked to the medical record. Each medicine prescribed creates a row in PRESCRIPTION_DETAIL, referencing the MEDICINE catalog.
6	Laboratory Tests Flow: medical_record → lab_test If lab investigations are required, LAB_TEST records are created, ordered by the doctor, linked to the patient and optionally to the medical record. Results are updated when ready.
7	Inpatient Admission Flow: patient → admission ← room ← doctor If the patient requires hospitalization, an ADMISSION record is created linking the patient, an available ROOM, and the responsible DOCTOR. Admission and discharge datetimes are tracked.

8

Billing & Payment*Flow: admission → billing ← patient*

On or after discharge, a BILLING record is generated from the admission. Total amount is calculated from room charges and services. Payments are tracked via paid_amount and payment_status.

9

Staff & Nurse Monitoring*Flow: staff → nurse → admission*

Hospital STAFF are assigned to departments. Nursing staff extend STAFF via the NURSE table and are assigned to monitor admitted patients.

3.5 CSV File Export (One Per Table)

All 14 CSV files have been generated and are included in the GitHub repository under the **/data/csv/** folder. Each file uses comma-separated format with a header row.

CSV File	Rows	Columns
department.csv	15 rows	department_id, department_name, location, head_doctor_id, phone
doctor.csv	40 rows	doctor_id, first_name, last_name, specialization, phone, email, department_id, hire_date, license_number
patient.csv	100 rows	patient_id, first_name, last_name, date_of_birth, gender, blood_type, phone, email, address, registration_date
staff.csv	50 rows	staff_id, first_name, last_name, role, department_id, phone, email, hire_date
nurse.csv	25 rows	nurse_id, staff_id, shift, department_id, certification
room.csv	40 rows	room_id, room_number, room_type, department_id, status, price_per_day
appointment.csv	200 rows	appointment_id, patient_id, doctor_id, appointment_date, status, reason, notes
medical_record.csv	120 rows	record_id, patient_id, doctor_id, visit_date, diagnosis, treatment, notes, appointment_id
prescription.csv	120 rows	prescription_id, record_id, patient_id, prescribed_date, expiry_date, status
medicine.csv	50 rows	medicine_id, medicine_name, generic_name, category, manufacturer, stock_quantity, unit_price
prescription_detail.csv	228 rows	detail_id, prescription_id, medicine_id, dosage, frequency, duration_days, instructions
admission.csv	60 rows	admission_id, patient_id, room_id, doctor_id, admission_date, discharge_date, diagnosis, status
billing.csv	60 rows	bill_id, patient_id, admission_id, total_amount, paid_amount, payment_status, bill_date, payment_method
lab_test.csv	100 rows	test_id, patient_id, doctor_id, test_name, test_date, results, status, record_id

Milestone 4

CREATE TABLE DDL Statements

All 14 tables with PK, FK, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints

Milestone 4 — DDL Scripts with All Constraints

The complete CREATE TABLE script (hms_milestone4_5.sql) is committed to the GitHub repository. Below is the schema for each table with every constraint documented.

4.1 Constraint Types Used

PRIMARY KEY

Every table has a single-column INT AUTO_INCREMENT PK. Guarantees row uniqueness and fast index lookups.

FOREIGN KEY

All inter-table references use FK constraints with explicit ON UPDATE CASCADE and ON DELETE (CASCADE/RESTRICT/SET NULL) actions.

NOT NULL

Applied to all columns that must always have a value — names, dates, status fields, phone numbers.

UNIQUE

Applied to natural identifiers: phone numbers, email addresses, room numbers, doctor license numbers.

CHECK

Domain validation: gender IN ('Male','Female','Other'), price > 0, stock >= 0, date ranges, status enumerations.

DEFAULT

Status fields default to their initial state (e.g., DEFAULT 'Available', DEFAULT 'Scheduled', DEFAULT 0.00).

4.2 Table Schemas with Constraints

DEPARTMENT			
Column	Data Type	Constraint	Description
department_id	INT	PK, AUTO_INCREMENT	Surrogate primary key
department_name	VARCHAR(100)	NOT NULL, UNIQUE	e.g. Cardiology
location	VARCHAR(100)	NOT NULL	Physical floor/block location
head_doctor_id	INT	FK → doctor, SET NULL	Department head doctor
phone	VARCHAR(15)	NULL, CHECK(format)	Department phone number
DOCTOR			

Column	Data Type	Constraint	Description
doctor_id	INT	PK, AUTO_INCREMENT	Surrogate PK
first_name	VARCHAR(50)	NOT NULL	Doctor first name
last_name	VARCHAR(50)	NOT NULL	Doctor last name
specialization	VARCHAR(100)	NOT NULL	Medical specialty
phone	VARCHAR(15)	NOT NULL, UNIQUE	Contact number
email	VARCHAR(100)	NULL, UNIQUE, CHECK(@)	Email address
department_id	INT	FK → department, RESTRICT	Owning department
hire_date	DATE	NOT NULL, CHECK(≥2000)	Employment start date
license_number	VARCHAR(50)	NOT NULL, UNIQUE	PMC license ID

PATIENT			
Column	Data Type	Constraint	Description
patient_id	INT	PK, AUTO_INCREMENT	Surrogate PK
first_name	VARCHAR(50)	NOT NULL	Patient first name
last_name	VARCHAR(50)	NOT NULL	Patient last name
date_of_birth	DATE	NOT NULL, CHECK(≤TODAY)	Date of birth
gender	VARCHAR(10)	NOT NULL, CHECK(IN list)	Male/Female/Other
blood_type	VARCHAR(5)	NULL, CHECK(IN list)	e.g. A+, O-
phone	VARCHAR(15)	NOT NULL, UNIQUE	Contact phone
email	VARCHAR(100)	NULL	Email address
address	TEXT	NULL	Residential address
registration_date	DATE	NOT NULL, DEFAULT TODAY	First visit date

STAFF			
Column	Data Type	Constraint	Description
staff_id	INT	PK, AUTO_INCREMENT	Surrogate PK
first_name	VARCHAR(50)	NOT NULL	Staff first name
last_name	VARCHAR(50)	NOT NULL	Staff last name
role	VARCHAR(100)	NOT NULL	Job role/title
department_id	INT	FK → department, RESTRICT	Assigned department
phone	VARCHAR(15)	NOT NULL, UNIQUE	Contact phone

email	VARCHAR(100)	NULL	Email address
hire_date	DATE	NOT NULL, CHECK(≥2000)	Employment date

NURSE			
Column	Data Type	Constraint	Description
nurse_id	INT	PK, AUTO_INCREMENT	Surrogate PK
staff_id	INT	FK → staff, CASCADE, UNIQUE	Links to STAFF (IS-A)
shift	VARCHAR(20)	NOT NULL, CHECK(IN list)	Morning/Evening/Night
department_id	INT	FK → department, RESTRICT	Assigned department
certification	VARCHAR(100)	NULL	Nursing certification

ROOM			
Column	Data Type	Constraint	Description
room_id	INT	PK, AUTO_INCREMENT	Surrogate PK
room_number	VARCHAR(20)	NOT NULL, UNIQUE	e.g. R-101
room_type	VARCHAR(50)	NOT NULL, CHECK(IN list)	General/Private/ICU...
department_id	INT	FK → department, RESTRICT	Owning department
status	VARCHAR(20)	NOT NULL, CHECK(IN list), DEFAULT 'Available'	Availability status
price_per_day	DECIMAL(10,2)	NOT NULL, CHECK(>0)	Daily charge (PKR)

APPOINTMENT			
Column	Data Type	Constraint	Description
appointment_id	INT	PK, AUTO_INCREMENT	Surrogate PK
patient_id	INT	FK → patient, CASCADE	Booked patient
doctor_id	INT	FK → doctor, RESTRICT	Attending doctor
appointment_date	DATETIME	NOT NULL	Scheduled date and time
status	VARCHAR(20)	NOT NULL, CHECK(IN list), DEFAULT 'Scheduled'	Visit status
reason	TEXT	NULL	Chief complaint
notes	TEXT	NULL	Additional notes

MEDICAL_RECORD			
Column	Data Type	Constraint	Description
record_id	INT	PK, AUTO_INCREMENT	Surrogate PK
patient_id	INT	FK → patient, CASCADE	Examined patient
doctor_id	INT	FK → doctor, RESTRICT	Treating doctor
visit_date	DATE	NOT NULL, CHECK(≥2000)	Date of clinical visit
diagnosis	TEXT	NOT NULL	Clinical diagnosis text
treatment	TEXT	NULL	Treatment plan
notes	TEXT	NULL	Clinical observations
appointment_id	INT	FK → appointment, SET NULL	Source appointment

PRESCRIPTION			
Column	Data Type	Constraint	Description
prescription_id	INT	PK, AUTO_INCREMENT	Surrogate PK
record_id	INT	FK → medical_record, CASCADE	Source record
patient_id	INT	FK → patient, CASCADE	Patient reference
prescribed_date	DATE	NOT NULL	Date of prescription
expiry_date	DATE	NULL, CHECK(≥prescribed_date)	Expiry date
status	VARCHAR(20)	NOT NULL, CHECK(IN list), DEFAULT 'Active'	Active/Dispensed/Expired

MEDICINE			
Column	Data Type	Constraint	Description
medicine_id	INT	PK, AUTO_INCREMENT	Surrogate PK
medicine_name	VARCHAR(150)	NOT NULL	Brand name
generic_name	VARCHAR(150)	NOT NULL	Generic/scientific name
category	VARCHAR(100)	NOT NULL	Drug category
manufacturer	VARCHAR(100)	NULL	Manufacturing company
stock_quantity	INT	NOT NULL, CHECK(≥0), DEFAULT 0	Current stock units

unit_price	DECIMAL(10,2)	NOT NULL, CHECK(>0)	Price per unit (PKR)
------------	---------------	------------------------	----------------------

PRESCRIPTION_DETAIL			
Column	Data Type	Constraint	Description
detail_id	INT	PK, AUTO_INCREMENT	Surrogate PK
prescription_id	INT	FK → prescription, CASCADE	Parent prescription
medicine_id	INT	FK → medicine, RESTRICT	Referenced medicine
dosage	VARCHAR(50)	NOT NULL	e.g. 500mg
frequency	VARCHAR(50)	NOT NULL	e.g. Twice daily
duration_days	INT	NOT NULL, CHECK(>0)	Duration in days
instructions	TEXT	NULL	Special instructions

ADMISSION			
Column	Data Type	Constraint	Description
admission_id	INT	PK, AUTO_INCREMENT	Surrogate PK
patient_id	INT	FK → patient, CASCADE	Admitted patient
room_id	INT	FK → room, RESTRICT	Assigned room
doctor_id	INT	FK → doctor, RESTRICT	Responsible doctor
admission_date	DATETIME	NOT NULL	Admission timestamp
discharge_date	DATETIME	NULL, CHECK(≥admission_date)	Discharge timestamp
diagnosis	TEXT	NOT NULL	Reason for admission
status	VARCHAR(20)	NOT NULL, CHECK(IN list), DEFAULT 'Admitted'	Admitted/Discharged

BILLING			
Column	Data Type	Constraint	Description
bill_id	INT	PK, AUTO_INCREMENT	Surrogate PK
patient_id	INT	FK → patient, RESTRICT	Billed patient
admission_id	INT	FK → admission, SET NULL	Source admission (optional)
total_amount	DECIMAL(12,2)	NOT NULL, CHECK(>0)	Invoice total (PKR)

paid_amount	DECIMAL(12,2)	NOT NULL, CHECK($\geq 0 \leq \text{total}$), DEFAULT 0	Amount paid so far
payment_status	VARCHAR(20)	NOT NULL, CHECK(IN list), DEFAULT 'Unpaid'	Unpaid/Partial/Paid
bill_date	DATE	NOT NULL	Date of invoice
payment_method	VARCHAR(50)	NULL, CHECK(IN list)	Cash/Card/Insurance/Online

LAB_TEST			
Column	Data Type	Constraint	Description
test_id	INT	PK, AUTO_INCREMENT	Surrogate PK
patient_id	INT	FK → patient, CASCADE	Tested patient
doctor_id	INT	FK → doctor, RESTRICT	Ordering doctor
test_name	VARCHAR(150)	NOT NULL	e.g. Complete Blood Count
test_date	DATE	NOT NULL	Date of test
results	TEXT	NULL	Result details
status	VARCHAR(20)	NOT NULL, CHECK(IN list), DEFAULT 'Pending'	Pending/Completed
record_id	INT	FK → medical_record, SET NULL	Linked clinical record

4.3 How to Run in MySQL Workbench

- Step 1: Download **hms_milestone4_5.sql** from the GitHub repository.
- Step 2: Open MySQL Workbench and connect to your local MySQL 8.0 instance.
- Step 3: Go to **File** → **Open SQL Script...** and select the downloaded .sql file.
- Step 4: Press **Ctrl + Shift + Enter** to execute all statements.
- Step 5: Verify in the Schemas panel: **hospital_db** should appear with all 14 tables.
- Step 6: To view the ERD: **Database** → **Reverse Engineer...** → select hospital_db → Finish.
- Step 7: Export the EER diagram: **File** → **Export** → **Export as PNG...**

Milestone 5

Data Population & Validation Queries

1,158 records inserted via INSERT statements — 12 validation queries with expected results

Milestone 5 — Database Population & Validation

5.1 Population Method

Data has been inserted using **INSERT INTO ... VALUES** statements embedded in the hms_milestone4_5.sql file. The INSERT blocks follow the exact table creation order to satisfy all foreign key dependencies. SET FOREIGN_KEY_CHECKS = 0 is used during bulk insert for performance, then re-enabled immediately after.

Insertion Order: department → doctor → patient → staff → nurse → room → appointment → medical_record → prescription → medicine → prescription_detail → admission → billing → lab_test

This order ensures all parent tables exist before child tables reference them.

5.2 Row Counts Per Table

Table	Rows Inserted	Notes
department	15	15 real hospital departments
doctor	40	2-3 doctors per department
patient	100	Core patient registry
staff	50	10 role types, across 15 depts
nurse	25	25 staff members designated as nurses
room	40	5 room types, realistic PKR pricing
appointment	200	60% completed, 30% scheduled, 10% cancelled
medical_record	120	Only from completed appointments
prescription	120	One-to-one with medical records sampled
medicine	50	Real Pakistani hospital formulary drugs
prescription_detail	228	Avg 1.9 medicines per prescription
admission	60	80% discharged, 20% still admitted
billing	60	Full billing cycle records
lab_test	100	16 distinct lab test types
TOTAL	1,158	Across all 14 tables

5.3 Validation Queries

The following 12 validation queries are embedded at the end of hms_milestone4_5.sql. Run them after population to confirm data integrity, FK consistency, and business rule compliance.

#	Validation Query	Purpose	Expected Result
V 1	SELECT COUNT(*) per table	Confirm row counts match expected values	14 rows in result set
V 2	NULL checks on NOT NULL cols	Find any missing required values	All counts = 0
V 3	Orphan appointments check	FK: appointments → valid patient	COUNT = 0
V 4	Orphan medical records check	FK: records → valid patient	COUNT = 0
V 5	Billing paid > total check	CHECK: paid_amount ≤ total_amount	COUNT = 0
V 6	Billing by payment_status	Revenue summary grouped by status	3 rows: Paid/Partial/Unpaid
V 7	Top 10 patients by admissions	Most frequently admitted patients	10 rows, ordered DESC
V 8	Top 10 medicines prescribed	Most commonly prescribed drugs	10 rows, ordered DESC
V 9	Top 10 busiest doctors	Doctors with most appointments	10 rows, ordered DESC
V 10	Lab test completion rate	Status distribution with %	2-3 rows: Pending/Completed
V 11	Revenue summary	Gross / collected / outstanding	1 row with totals
V 12	Doctor count per department	Staffing by department	15 rows, ordered DESC

5.4 Sample Validation Output (Expected)

After running all INSERT statements, the validation queries should produce results similar to the following:

V6 — Billing by Payment Status

payment_status	bills	total_billed	total_paid	outstanding
Paid	~30	~2,400,000	~2,400,000	0
Partial	~20	~1,600,000	~800,000	~800,000
Unpaid	~10	~800,000	0	~800,000

V10 — Lab Test Completion Rate

status	count	pct
Completed	~50	~50%
Pending	~50	~50%

V11 — Revenue Summary

gross_revenue	collected	outstanding	collection_rate_pct
~4,800,000	~3,200,000	~1,600,000	~66.67

Appendix A — GitHub Repository Structure

All milestone deliverables must be committed to a GitHub repository. Each milestone should have its own commit with a clear message. The repository structure below is recommended:

```
hospital-management-system/
├── README.md
├── docs/
│   └── GroupName_Version_Control_Doc_DBLab.pdf
├── erd/
│   ├── hms_erd_screenshot.png ← MySQL Workbench EER export
│   └── hms_erd_mermaid.md ← Mermaid source
├── milestone2_normalization/
│   ├── normalization_analysis.md
│   └── updated_erd.png
├── milestone3_data/
│   ├── generate_data.py ← Faker data generation script
│   └── csv/
│       ├── department.csv (15 rows)
│       ├── doctor.csv (40 rows)
│       ├── patient.csv (100 rows)
│       └── ... (14 files total)
├── milestone4_ddl/
│   └── hms_create_tables.sql ← DDL with all constraints
├── milestone5_population/
│   └── hms_milestone4_5.sql ← DDL + INSERT + Validation
└── validation_results_screenshot.png
```

Recommended Git Commit Messages

Milestone	Git Commit Message
Milestone 1	feat: add ERD and initial schema description for HMS
Milestone 2	feat: add normalization analysis (1NF-3NF) for all 14 tables
Milestone 3	feat: add Faker data generation script and 14 CSV files (1158 rows)
Milestone 4	feat: add DDL CREATE TABLE statements with all constraints
Milestone 5	feat: add INSERT statements and 12 validation queries
Final	docs: add final PDF submission GroupName_Version_Control_Doc_DBLab.pdf

Appendix B — Entity List Summary

#	Table	Module	PK	FKs	Rows	Key Constraints
1	department	Organization	department_id	1 (doctor)	15	UNIQUE name, CHECK phone
2	doctor	Staff	doctor_id	1 (dept)	40	UNIQUE phone/license/email, CHECK hire
3	patient	Patient	patient_id	—	100	UNIQUE phone, CHECK gender/blood/dob
4	staff	HR	staff_id	1 (dept)	50	UNIQUE phone, CHECK hire
5	nurse	HR	nurse_id	2 (staff, dept)	25	UNIQUE staff_id, CHECK shift
6	room	Inpatient	room_id	1 (dept)	40	UNIQUE number, CHECK type/status/price
7	appointment	Scheduling	appointment_id	2 (patient, doctor)	200	CHECK status
8	medical_record	Clinical	record_id	3 (patient, doctor, appt)	120	CHECK visit date
9	prescription	Pharmacy	prescription_id	2 (record, patient)	120	CHECK status/expiry
10	medicine	Pharmacy	medicine_id	—	50	CHECK stock $\geq$ 0, price $>$ 0
11	prescription_detail	Pharmacy	detail_id	2 (rx, medicine)	228	CHECK duration $>$ 0
12	admission	Inpatient	admission_id	3 (patient, room, doctor)	60	CHECK discharge $\geq$ admission
13	billing	Finance	bill_id	2 (patient, admission)	60	CHECK paid $\leq$ total, status
14	lab_test	Lab	test_id	3 (patient, doctor, record)	100	CHECK status

Hospital Management System — Database Project | Milestones 2–5

Submitted for: Database Management Systems (DBMS) | Deadline: 17 May 2026